

st125333_LabAssignment8_DLCV

November 20, 2025

Name: Sat Naing Tun, ID: st125333

0.1 Exercise 1

```
[1]: import torch
import torch.nn as nn

class VanillaGenerator(nn.Module):
    def __init__(self, noise_dim=100):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(noise_dim, 256),
            nn.LeakyReLU(0.2),
            nn.Linear(256, 512),
            nn.LeakyReLU(0.2),
            nn.Linear(512, 784),
            nn.Tanh(),
        )

    def forward(self, z):
        return self.net(z).view(-1, 1, 28, 28)

class VanillaDiscriminator(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Flatten(),
            nn.Linear(784, 512),
            nn.LeakyReLU(0.2),
            nn.Linear(512, 256),
            nn.LeakyReLU(0.2),
            nn.Linear(256, 1),
            nn.Sigmoid(),
        )
```

```
def forward(self, x):
    return self.net(x)
```

```
[2]: import torch
import torch.nn as nn

class DCGANGenerator(nn.Module):
    def __init__(self, noise_dim=100, ngf=64):
        super().__init__()
        self.model = nn.Sequential(
            nn.ConvTranspose2d(noise_dim, ngf * 4, 4, 1, 0, bias=False),
            nn.BatchNorm2d(ngf * 4),
            nn.ReLU(True),

            nn.ConvTranspose2d(ngf * 4, ngf * 2, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf * 2),
            nn.ReLU(True),

            nn.ConvTranspose2d(ngf * 2, ngf, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ngf),
            nn.ReLU(True),

            nn.ConvTranspose2d(ngf, 3, 4, 2, 1, bias=False),
            nn.Tanh()
        )

    def forward(self, z):
        return self.model(z.view(z.size(0), z.size(1), 1, 1))
```

```
[3]: class DCGANDiscriminator(nn.Module):
    def __init__(self, ndf=64):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, ndf, 4, 2, 1, bias=False),
            nn.LeakyReLU(0.2, inplace=True),

            nn.Conv2d(ndf, ndf * 2, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf * 2),
            nn.LeakyReLU(0.2, inplace=True),

            nn.Conv2d(ndf * 2, ndf * 4, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf * 4),
            nn.LeakyReLU(0.2, inplace=True),
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
```

```

        nn.Linear(ndf * 4 * 4 * 4, 1), # adjust dims depending on input
↪size
        nn.Sigmoid()
    )

    def forward(self, x):
        x = self.features(x)
        return self.classifier(x) # shape [B, 1]

```

```

[4]: from torchvision import datasets, transforms
from torch.utils.data import DataLoader

def get_mnist(batch=128):
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.5,), (0.5,))
    ])
    train = datasets.MNIST("data", train=True, download=True,
↪transform=transform)
    return DataLoader(train, batch_size=batch, shuffle=True)

def get_cifar(batch=128):
    transform = transforms.Compose([
        transforms.Resize(32),
        transforms.CenterCrop(32),
        transforms.ToTensor(),
        transforms.Normalize((0.5,), (0.5,))
    ])
    train = datasets.CIFAR10("data", train=True, download=True,
↪transform=transform)
    return DataLoader(train, batch_size=batch, shuffle=True)

```

```

[5]: def train_one_epoch(generator, discriminator, loader, g_opt, d_opt, criterion,
↪epoch, device, noise_dim=100):
    generator.train()
    discriminator.train()

    g_loss_epoch = 0
    d_loss_epoch = 0

    for real_imgs, _ in loader:
        real_imgs = real_imgs.to(device)
        batch_size = real_imgs.size(0)

        # -----
        # 1. Train Discriminator
        # -----

```

```

noise = torch.randn(batch_size, noise_dim, device=device)
fake_imgs = generator(noise).detach()

real_labels = torch.ones(batch_size, 1, device=device)
fake_labels = torch.zeros(batch_size, 1, device=device)

d_real = discriminator(real_imgs)
d_fake = discriminator(fake_imgs)

d_loss_real = criterion(d_real, real_labels)
d_loss_fake = criterion(d_fake, fake_labels)
d_loss = d_loss_real + d_loss_fake

d_opt.zero_grad()
d_loss.backward()
d_opt.step()

# -----
# 2. Train Generator
# -----
noise = torch.randn(batch_size, noise_dim, device=device)
fake_imgs = generator(noise)

g_loss = criterion(discriminator(fake_imgs), real_labels)

g_opt.zero_grad()
g_loss.backward()
g_opt.step()

g_loss_epoch += g_loss.item()
d_loss_epoch += d_loss.item()

return g_loss_epoch / len(loader), d_loss_epoch / len(loader)

```

```

[6]: def validate_one_epoch(generator, discriminator, loader, criterion, device,
    ↪noise_dim=100):
    generator.eval()
    discriminator.eval()

    g_loss_epoch = 0
    d_loss_epoch = 0

    with torch.no_grad():
        for real_imgs, _ in loader:
            real_imgs = real_imgs.to(device)
            batch = real_imgs.size(0)

```

```

        noise = torch.randn(batch, noise_dim, device=device)
        fake_imgs = generator(noise)

        real_labels = torch.ones(batch, 1, device=device)
        fake_labels = torch.zeros(batch, 1, device=device)

        d_real = discriminator(real_imgs)
        d_fake = discriminator(fake_imgs)

        d_loss = criterion(d_real, real_labels) + criterion(d_fake,
↪fake_labels)
        g_loss = criterion(d_fake, real_labels)

        g_loss_epoch += g_loss.item()
        d_loss_epoch += d_loss.item()

    return g_loss_epoch/len(loader), d_loss_epoch/len(loader)

```

```

[7]: def train_gan(generator, discriminator, loader, epochs, noise_dim, device):
    criterion = nn.BCELoss()
    g_opt = torch.optim.Adam(generator.parameters(), lr=2e-4, betas=(0.5, 0.
↪999))
    d_opt = torch.optim.Adam(discriminator.parameters(), lr=2e-4, betas=(0.5, 0.
↪999))

    train_g_losses, train_d_losses = [], []
    val_g_losses, val_d_losses = [], []

    for epoch in range(1, epochs+1):
        g_train, d_train = train_one_epoch(
            generator, discriminator, loader,
            g_opt, d_opt, criterion, epoch, device,
            noise_dim=noise_dim
        )

        g_val, d_val = validate_one_epoch(
            generator, discriminator, loader,
            criterion, device, noise_dim=noise_dim
        )

        train_g_losses.append(g_train)
        train_d_losses.append(d_train)
        val_g_losses.append(g_val)
        val_d_losses.append(d_val)

        print(f"[Epoch {epoch}/{epochs}] G_train={g_train:.4f} D_train={d_train:
↪.4f} | G_val={g_val:.4f} D_val={d_val:.4f}")

```

```
return train_g_losses, train_d_losses, val_g_losses, val_d_losses
```

```
[8]: import matplotlib.pyplot as plt

def plot_gan_losses(g_train, d_train, g_val, d_val, title="GAN Loss"):
    plt.figure(figsize=(10,6))
    plt.plot(g_train, label="G Train")
    plt.plot(d_train, label="D Train")
    plt.plot(g_val, '--', label="G Val")
    plt.plot(d_val, '--', label="D Val")
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.title(title)
    plt.legend()
    plt.grid()
    plt.show()
```

```
[9]: device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(" Using device:", device)
```

Using device: cuda

```
[10]: # device = "cuda"

loader = get_mnist(batch=128)
G = VanillaGenerator().to(device)
D = VanillaDiscriminator().to(device)

g_tr, d_tr, g_val, d_val = train_gan(G, D, loader,
                                     epochs=50,
                                     noise_dim=100,
                                     device=device)
```

100%| | 9.91M/9.91M [00:01<00:00, 4.96MB/s]

100%| | 28.9k/28.9k [00:00<00:00, 132kB/s]

100%| | 1.65M/1.65M [00:01<00:00, 1.24MB/s]

100%| | 4.54k/4.54k [00:00<00:00, 8.29MB/s]

[Epoch 1/50] G_train=1.5415 D_train=0.8477 | G_val=2.5747 D_val=1.5991

[Epoch 2/50] G_train=2.3504 D_train=0.6795 | G_val=0.5492 D_val=1.0270

[Epoch 3/50] G_train=2.5753 D_train=0.6296 | G_val=1.7324 D_val=0.3650

[Epoch 4/50] G_train=2.4673 D_train=0.6683 | G_val=1.3039 D_val=0.6065

[Epoch 5/50] G_train=2.3866 D_train=0.6181 | G_val=1.3717 D_val=0.4954

[Epoch 6/50] G_train=2.5838 D_train=0.5758 | G_val=1.9456 D_val=0.5170

[Epoch 7/50] G_train=2.8963 D_train=0.5548 | G_val=2.6192 D_val=0.2891

[Epoch 8/50] G_train=2.9977 D_train=0.5957 | G_val=1.7312 D_val=0.4884

[Epoch 9/50] G_train=2.6873 D_train=0.5869 | G_val=1.5030 D_val=0.4563

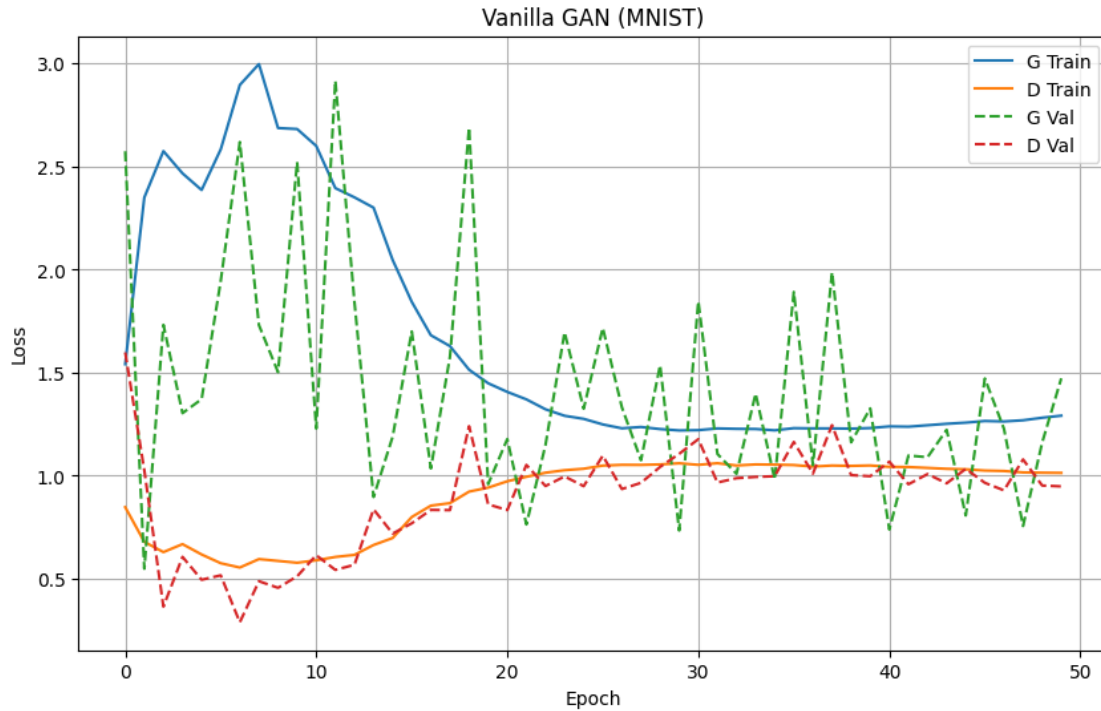
[Epoch 10/50] G_train=2.6825 D_train=0.5781 | G_val=2.5224 D_val=0.5121

```

[Epoch 11/50] G_train=2.6003 D_train=0.5899 | G_val=1.2291 D_val=0.6148
[Epoch 12/50] G_train=2.3958 D_train=0.6061 | G_val=2.9169 D_val=0.5432
[Epoch 13/50] G_train=2.3510 D_train=0.6162 | G_val=1.8428 D_val=0.5681
[Epoch 14/50] G_train=2.3010 D_train=0.6641 | G_val=0.8969 D_val=0.8374
[Epoch 15/50] G_train=2.0472 D_train=0.6980 | G_val=1.1897 D_val=0.7194
[Epoch 16/50] G_train=1.8441 D_train=0.7999 | G_val=1.7002 D_val=0.7694
[Epoch 17/50] G_train=1.6819 D_train=0.8550 | G_val=1.0356 D_val=0.8344
[Epoch 18/50] G_train=1.6291 D_train=0.8673 | G_val=1.5776 D_val=0.8333
[Epoch 19/50] G_train=1.5147 D_train=0.9230 | G_val=2.6892 D_val=1.2407
[Epoch 20/50] G_train=1.4489 D_train=0.9425 | G_val=0.9592 D_val=0.8613
[Epoch 21/50] G_train=1.4069 D_train=0.9733 | G_val=1.1780 D_val=0.8335
[Epoch 22/50] G_train=1.3707 D_train=0.9950 | G_val=0.7637 D_val=1.0527
[Epoch 23/50] G_train=1.3225 D_train=1.0149 | G_val=1.1493 D_val=0.9502
[Epoch 24/50] G_train=1.2911 D_train=1.0270 | G_val=1.6963 D_val=0.9968
[Epoch 25/50] G_train=1.2762 D_train=1.0341 | G_val=1.3252 D_val=0.9495
[Epoch 26/50] G_train=1.2491 D_train=1.0494 | G_val=1.7175 D_val=1.0976
[Epoch 27/50] G_train=1.2303 D_train=1.0531 | G_val=1.3322 D_val=0.9356
[Epoch 28/50] G_train=1.2367 D_train=1.0527 | G_val=1.0754 D_val=0.9658
[Epoch 29/50] G_train=1.2266 D_train=1.0551 | G_val=1.5365 D_val=1.0430
[Epoch 30/50] G_train=1.2200 D_train=1.0606 | G_val=0.7345 D_val=1.1066
[Epoch 31/50] G_train=1.2214 D_train=1.0531 | G_val=1.8467 D_val=1.1776
[Epoch 32/50] G_train=1.2298 D_train=1.0609 | G_val=1.1060 D_val=0.9672
[Epoch 33/50] G_train=1.2277 D_train=1.0500 | G_val=1.0095 D_val=0.9883
[Epoch 34/50] G_train=1.2269 D_train=1.0550 | G_val=1.3981 D_val=0.9943
[Epoch 35/50] G_train=1.2205 D_train=1.0540 | G_val=0.9823 D_val=0.9978
[Epoch 36/50] G_train=1.2310 D_train=1.0521 | G_val=1.8918 D_val=1.1643
[Epoch 37/50] G_train=1.2300 D_train=1.0446 | G_val=1.0519 D_val=1.0104
[Epoch 38/50] G_train=1.2298 D_train=1.0497 | G_val=1.9875 D_val=1.2455
[Epoch 39/50] G_train=1.2287 D_train=1.0480 | G_val=1.1624 D_val=1.0032
[Epoch 40/50] G_train=1.2315 D_train=1.0500 | G_val=1.3270 D_val=0.9978
[Epoch 41/50] G_train=1.2396 D_train=1.0433 | G_val=0.7391 D_val=1.0684
[Epoch 42/50] G_train=1.2382 D_train=1.0423 | G_val=1.0983 D_val=0.9588
[Epoch 43/50] G_train=1.2452 D_train=1.0382 | G_val=1.0901 D_val=1.0083
[Epoch 44/50] G_train=1.2522 D_train=1.0338 | G_val=1.2231 D_val=0.9634
[Epoch 45/50] G_train=1.2578 D_train=1.0314 | G_val=0.8074 D_val=1.0336
[Epoch 46/50] G_train=1.2658 D_train=1.0253 | G_val=1.4720 D_val=0.9659
[Epoch 47/50] G_train=1.2635 D_train=1.0231 | G_val=1.2282 D_val=0.9292
[Epoch 48/50] G_train=1.2693 D_train=1.0164 | G_val=0.7562 D_val=1.0793
[Epoch 49/50] G_train=1.2816 D_train=1.0148 | G_val=1.1609 D_val=0.9527
[Epoch 50/50] G_train=1.2916 D_train=1.0140 | G_val=1.4728 D_val=0.9485

```

```
[11]: plot_gan_losses(g_tr, d_tr, g_val, d_val, title="Vanilla GAN (MNIST)")
```



```
[12]: device = "cuda"

loader = get_cifar(batch=128)
G = DCGANGenerator().to(device)
D = DCGANDiscriminator().to(device)

g_tr, d_tr, g_val, d_val = train_gan(G, D, loader,
                                     epochs=100,
                                     noise_dim=100,
                                     device=device)
```

100% | 170M/170M [00:14<00:00, 11.8MB/s]

```
[Epoch 1/100] G_train=4.7127 D_train=0.3673 | G_val=2.0086 D_val=0.9475
[Epoch 2/100] G_train=3.7223 D_train=0.4553 | G_val=2.3815 D_val=1.0401
[Epoch 3/100] G_train=2.9081 D_train=0.5869 | G_val=1.2700 D_val=1.1776
[Epoch 4/100] G_train=2.7574 D_train=0.5556 | G_val=1.0935 D_val=0.9641
[Epoch 5/100] G_train=2.5741 D_train=0.6140 | G_val=2.0406 D_val=1.2636
[Epoch 6/100] G_train=2.6738 D_train=0.5940 | G_val=0.9474 D_val=1.0552
[Epoch 7/100] G_train=2.7146 D_train=0.5542 | G_val=1.3630 D_val=0.8687
[Epoch 8/100] G_train=2.7165 D_train=0.5313 | G_val=1.7796 D_val=0.8452
[Epoch 9/100] G_train=2.6512 D_train=0.5187 | G_val=2.9255 D_val=1.5539
[Epoch 10/100] G_train=2.7477 D_train=0.4921 | G_val=1.6778 D_val=0.7710
[Epoch 11/100] G_train=2.7087 D_train=0.5574 | G_val=1.6050 D_val=0.8293
[Epoch 12/100] G_train=2.7641 D_train=0.5125 | G_val=1.7888 D_val=1.0464
```

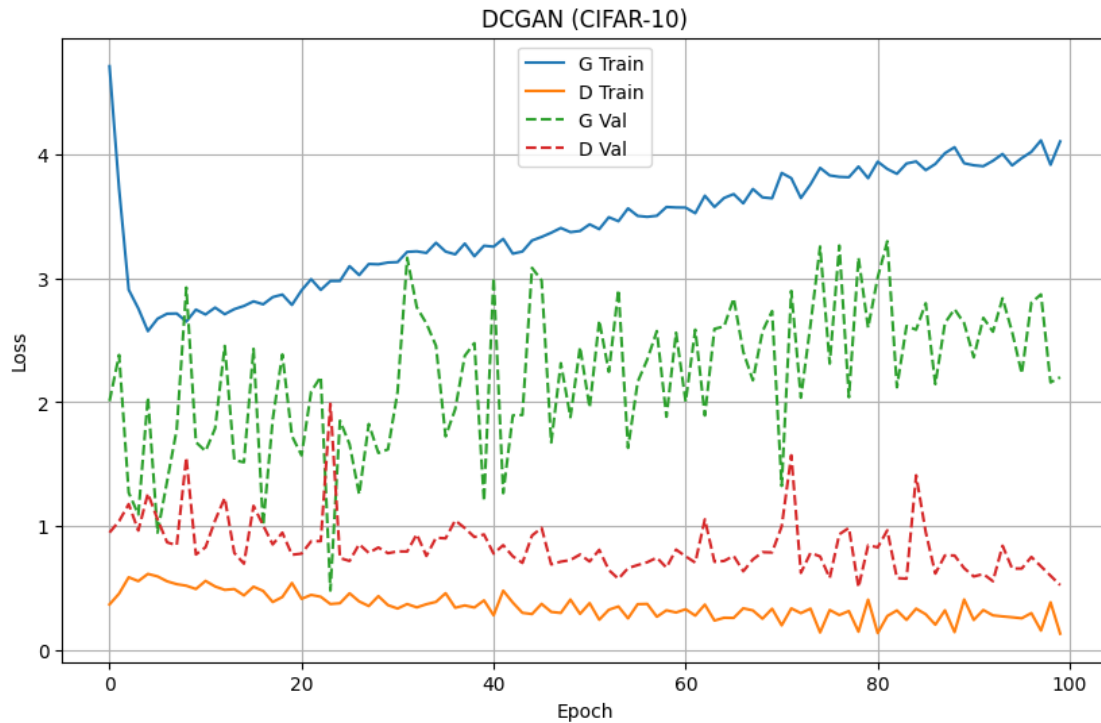
[Epoch 13/100] G_train=2.7108 D_train=0.4852 | G_val=2.4543 D_val=1.2280
 [Epoch 14/100] G_train=2.7515 D_train=0.4916 | G_val=1.5318 D_val=0.7829
 [Epoch 15/100] G_train=2.7772 D_train=0.4403 | G_val=1.5154 D_val=0.6969
 [Epoch 16/100] G_train=2.8145 D_train=0.5106 | G_val=2.4310 D_val=1.1636
 [Epoch 17/100] G_train=2.7905 D_train=0.4765 | G_val=1.0060 D_val=1.0063
 [Epoch 18/100] G_train=2.8487 D_train=0.3872 | G_val=1.8632 D_val=0.8520
 [Epoch 19/100] G_train=2.8692 D_train=0.4275 | G_val=2.3856 D_val=0.9472
 [Epoch 20/100] G_train=2.7855 D_train=0.5410 | G_val=1.7272 D_val=0.7686
 [Epoch 21/100] G_train=2.9030 D_train=0.4126 | G_val=1.5710 D_val=0.7776
 [Epoch 22/100] G_train=2.9938 D_train=0.4446 | G_val=2.0858 D_val=0.8777
 [Epoch 23/100] G_train=2.9066 D_train=0.4294 | G_val=2.2055 D_val=0.8786
 [Epoch 24/100] G_train=2.9785 D_train=0.3709 | G_val=0.4800 D_val=1.9868
 [Epoch 25/100] G_train=2.9792 D_train=0.3772 | G_val=1.8514 D_val=0.7419
 [Epoch 26/100] G_train=3.0988 D_train=0.4575 | G_val=1.6610 D_val=0.7185
 [Epoch 27/100] G_train=3.0262 D_train=0.3918 | G_val=1.2558 D_val=0.8541
 [Epoch 28/100] G_train=3.1163 D_train=0.3546 | G_val=1.8244 D_val=0.7835
 [Epoch 29/100] G_train=3.1135 D_train=0.4347 | G_val=1.5905 D_val=0.8273
 [Epoch 30/100] G_train=3.1280 D_train=0.3618 | G_val=1.6192 D_val=0.7809
 [Epoch 31/100] G_train=3.1313 D_train=0.3344 | G_val=2.0839 D_val=0.7960
 [Epoch 32/100] G_train=3.2141 D_train=0.3707 | G_val=3.1667 D_val=0.7950
 [Epoch 33/100] G_train=3.2181 D_train=0.3437 | G_val=2.7660 D_val=0.9346
 [Epoch 34/100] G_train=3.2056 D_train=0.3692 | G_val=2.6367 D_val=0.7591
 [Epoch 35/100] G_train=3.2873 D_train=0.3881 | G_val=2.4554 D_val=0.9058
 [Epoch 36/100] G_train=3.2163 D_train=0.4584 | G_val=1.7241 D_val=0.9009
 [Epoch 37/100] G_train=3.1939 D_train=0.3410 | G_val=1.9430 D_val=1.0462
 [Epoch 38/100] G_train=3.2810 D_train=0.3595 | G_val=2.3754 D_val=0.9839
 [Epoch 39/100] G_train=3.1780 D_train=0.3426 | G_val=2.4773 D_val=0.9118
 [Epoch 40/100] G_train=3.2635 D_train=0.4003 | G_val=1.2003 D_val=0.9325
 [Epoch 41/100] G_train=3.2557 D_train=0.2797 | G_val=3.0062 D_val=0.7774
 [Epoch 42/100] G_train=3.3188 D_train=0.4784 | G_val=1.2659 D_val=0.8460
 [Epoch 43/100] G_train=3.1994 D_train=0.3825 | G_val=1.8935 D_val=0.7560
 [Epoch 44/100] G_train=3.2168 D_train=0.2999 | G_val=1.8972 D_val=0.7027
 [Epoch 45/100] G_train=3.3056 D_train=0.2900 | G_val=3.0849 D_val=0.9236
 [Epoch 46/100] G_train=3.3347 D_train=0.3720 | G_val=2.9830 D_val=0.9857
 [Epoch 47/100] G_train=3.3683 D_train=0.3072 | G_val=1.6755 D_val=0.6904
 [Epoch 48/100] G_train=3.4070 D_train=0.2989 | G_val=2.3148 D_val=0.7129
 [Epoch 49/100] G_train=3.3732 D_train=0.4070 | G_val=1.8792 D_val=0.7257
 [Epoch 50/100] G_train=3.3825 D_train=0.2911 | G_val=2.4462 D_val=0.7706
 [Epoch 51/100] G_train=3.4361 D_train=0.3784 | G_val=1.9565 D_val=0.7141
 [Epoch 52/100] G_train=3.3974 D_train=0.2441 | G_val=2.6600 D_val=0.8085
 [Epoch 53/100] G_train=3.4938 D_train=0.3246 | G_val=2.2455 D_val=0.6452
 [Epoch 54/100] G_train=3.4612 D_train=0.3511 | G_val=2.9090 D_val=0.5744
 [Epoch 55/100] G_train=3.5651 D_train=0.2547 | G_val=1.6314 D_val=0.6605
 [Epoch 56/100] G_train=3.5043 D_train=0.3695 | G_val=2.1636 D_val=0.6853
 [Epoch 57/100] G_train=3.4968 D_train=0.3715 | G_val=2.3447 D_val=0.7081
 [Epoch 58/100] G_train=3.5053 D_train=0.2678 | G_val=2.5747 D_val=0.7452
 [Epoch 59/100] G_train=3.5762 D_train=0.3202 | G_val=1.8826 D_val=0.6647
 [Epoch 60/100] G_train=3.5723 D_train=0.3025 | G_val=2.5615 D_val=0.8103

```

[Epoch 61/100] G_train=3.5713 D_train=0.3286 | G_val=2.0044 D_val=0.7555
[Epoch 62/100] G_train=3.5259 D_train=0.2772 | G_val=2.5859 D_val=0.7053
[Epoch 63/100] G_train=3.6673 D_train=0.3660 | G_val=1.8930 D_val=1.0546
[Epoch 64/100] G_train=3.5765 D_train=0.2373 | G_val=2.5922 D_val=0.7122
[Epoch 65/100] G_train=3.6490 D_train=0.2601 | G_val=2.6115 D_val=0.7185
[Epoch 66/100] G_train=3.6805 D_train=0.2588 | G_val=2.8385 D_val=0.7595
[Epoch 67/100] G_train=3.6056 D_train=0.3364 | G_val=2.4070 D_val=0.6353
[Epoch 68/100] G_train=3.7215 D_train=0.3190 | G_val=2.1766 D_val=0.7303
[Epoch 69/100] G_train=3.6540 D_train=0.2523 | G_val=2.5719 D_val=0.7895
[Epoch 70/100] G_train=3.6463 D_train=0.3329 | G_val=2.7359 D_val=0.7872
[Epoch 71/100] G_train=3.8512 D_train=0.1986 | G_val=1.3246 D_val=0.9985
[Epoch 72/100] G_train=3.8088 D_train=0.3355 | G_val=2.8991 D_val=1.5716
[Epoch 73/100] G_train=3.6485 D_train=0.2971 | G_val=2.0368 D_val=0.6228
[Epoch 74/100] G_train=3.7585 D_train=0.3335 | G_val=2.6257 D_val=0.7830
[Epoch 75/100] G_train=3.8920 D_train=0.1412 | G_val=3.2569 D_val=0.7554
[Epoch 76/100] G_train=3.8313 D_train=0.3227 | G_val=2.3122 D_val=0.5808
[Epoch 77/100] G_train=3.8197 D_train=0.2820 | G_val=3.2648 D_val=0.9341
[Epoch 78/100] G_train=3.8166 D_train=0.3141 | G_val=2.0406 D_val=0.9853
[Epoch 79/100] G_train=3.9028 D_train=0.1476 | G_val=3.1674 D_val=0.5055
[Epoch 80/100] G_train=3.8086 D_train=0.4062 | G_val=2.6015 D_val=0.8452
[Epoch 81/100] G_train=3.9421 D_train=0.1364 | G_val=3.0081 D_val=0.8282
[Epoch 82/100] G_train=3.8832 D_train=0.2742 | G_val=3.2991 D_val=0.9659
[Epoch 83/100] G_train=3.8441 D_train=0.3190 | G_val=2.1209 D_val=0.5801
[Epoch 84/100] G_train=3.9274 D_train=0.2429 | G_val=2.6161 D_val=0.5758
[Epoch 85/100] G_train=3.9437 D_train=0.3335 | G_val=2.5857 D_val=1.4098
[Epoch 86/100] G_train=3.8738 D_train=0.2881 | G_val=2.7992 D_val=0.9512
[Epoch 87/100] G_train=3.9235 D_train=0.2039 | G_val=2.1460 D_val=0.6168
[Epoch 88/100] G_train=4.0106 D_train=0.3201 | G_val=2.6394 D_val=0.7632
[Epoch 89/100] G_train=4.0588 D_train=0.1440 | G_val=2.7517 D_val=0.7629
[Epoch 90/100] G_train=3.9296 D_train=0.4074 | G_val=2.6375 D_val=0.6601
[Epoch 91/100] G_train=3.9129 D_train=0.2422 | G_val=2.3621 D_val=0.5931
[Epoch 92/100] G_train=3.9056 D_train=0.3225 | G_val=2.6814 D_val=0.6129
[Epoch 93/100] G_train=3.9491 D_train=0.2793 | G_val=2.5693 D_val=0.5550
[Epoch 94/100] G_train=4.0044 D_train=0.2708 | G_val=2.8392 D_val=0.8415
[Epoch 95/100] G_train=3.9118 D_train=0.2638 | G_val=2.5668 D_val=0.6571
[Epoch 96/100] G_train=3.9710 D_train=0.2558 | G_val=2.2320 D_val=0.6572
[Epoch 97/100] G_train=4.0214 D_train=0.2977 | G_val=2.8038 D_val=0.7509
[Epoch 98/100] G_train=4.1142 D_train=0.1566 | G_val=2.8704 D_val=0.6753
[Epoch 99/100] G_train=3.9171 D_train=0.3847 | G_val=2.1604 D_val=0.5990
[Epoch 100/100] G_train=4.1058 D_train=0.1317 | G_val=2.2010 D_val=0.5235

```

```
[13]: plot_gan_losses(g_tr, d_tr, g_val, d_val, title="DCGAN (CIFAR-10)")
```



0.2 Exercise 2

```
[14]: import os
import requests
from tqdm import tqdm

def download_big_file(dest_path,url):
    # Ensure destination directory exists
    os.makedirs(os.path.dirname(dest_path), exist_ok=True)

    if os.path.exists(dest_path):
        print(f" Skipping {os.path.basename(dest_path)} (already exists)")
        return

    response = requests.get(url, stream=True)
    total = int(response.headers.get('content-length', 0))

    with open(dest_path, 'wb') as file, tqdm(
        desc=os.path.basename(dest_path),
        total=total,
        unit='B',
        unit_scale=True,
        unit_divisor=1024,
```

```

    ) as bar:
        for data in response.iter_content(chunk_size=1024):
            size = file.write(data)
            bar.update(size)

```

```

[15]: import os
import zipfile
from tqdm import tqdm

def unzip_folder(zip_path, extract_to, sub_dir="train"):
    expected_path = os.path.join(extract_to, sub_dir)
    if os.path.exists(expected_path) and any(os.listdir(expected_path)):
        print(f" Skipping extraction: {expected_path} already populated")
        return

    print(f" Extracting {os.path.basename(zip_path)} → {extract_to}")
    with zipfile.ZipFile(zip_path, 'r') as zip_ref:
        for member in tqdm(zip_ref.infolist(), desc="Extracting", unit="file"):
            zip_ref.extract(member, extract_to)

```

```

[16]: # raw_zip_dir = "DLCV_Lab_Dataset/raw_zips/human-faces.zip"
# source_url = "https://www.kaggle.com/api/v1/datasets/download/ashwingupta3012/
↳human-faces"
# download_big_file(raw_zip_dir, source_url)

```

```

[17]: # zip_path = "DLCV_Lab_Dataset/raw_zips/human-faces.zip"
# extract_path = "DLCV_Lab_Dataset"
# unzip_folder(zip_path, extract_path)

```

```

[18]: raw_zip_dir = "DLCV_Lab_Dataset/raw_zips/human-faces-dataset.zip"
source_url = "https://www.kaggle.com/api/v1/datasets/download/kaustubhdhote/
↳human-faces-dataset"
download_big_file(raw_zip_dir, source_url)

```

```

human-faces-dataset.zip: 100%|          | 114M/114M [00:06<00:00, 19.1MB/s]

```

```

[19]: zip_path = "DLCV_Lab_Dataset/raw_zips/human-faces-dataset.zip"
extract_path = "DLCV_Lab_Dataset"
unzip_folder(zip_path, extract_path)

```

```

Extracting human-faces-dataset.zip → DLCV_Lab_Dataset

```

```

Extracting: 100%|          | 9630/9630 [00:01<00:00, 5982.28file/s]

```

```

[20]: from PIL import Image

image_path = '/content/DLCV_Lab_Dataset/Human Faces Dataset/Real Images/000001.
↳jpg'
img = Image.open(image_path)

```

```
width, height = img.size
print(f"Width: {width}, Height: {height}")
```

Width: 178, Height: 218

```
[21]: import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
from tqdm import tqdm
import os
```

```
[22]: from PIL import Image
import torchvision.transforms as T

class CustomImageFolder(torch.utils.data.Dataset):
    def __init__(self, root, target_size=(160, 160)):
        self.root = root
        self.paths = [
            os.path.join(root, f)
            for f in os.listdir(root)
            if f.lower().endswith(("jpg", "jpeg", "png"))
        ]

        self.transform = T.Compose([
            T.Resize(target_size),
            T.CenterCrop(target_size),
            T.ToTensor(),
            T.Normalize([0.5]*3, [0.5]*3),
        ])

    def __len__(self):
        return len(self.paths)

    def __getitem__(self, idx):
        img = Image.open(self.paths[idx]).convert("RGB")
        return self.transform(img)
```

```
[23]: class UniversalGenerator(nn.Module):
    def __init__(self, latent_dim=256, base_channels=64, target_size=(160,160),
        im_chan=3):
        super().__init__()
        H, W = target_size
```

```

assert H % 16 == 0 and W % 16 == 0, "Target size must be divisible by_
↪16."

```

```

layers = []
c = base_channels * 8

# Start from 4x4
layers.append(nn.Sequential(
    nn.ConvTranspose2d(latent_dim, c, 4, 1, 0),
    nn.BatchNorm2d(c),
    nn.ReLU(True)
))

out_h, out_w = 4, 4

# Upsample dynamically until reaching target size
while out_h < H or out_w < W:
    layers.append(nn.Sequential(
        nn.ConvTranspose2d(c, c // 2, 4, 2, 1),
        nn.BatchNorm2d(c // 2),
        nn.ReLU(True)
    ))
    c //= 2
    out_h *= 2
    out_w *= 2

layers.append(nn.ConvTranspose2d(c, im_chan, 3, 1, 1))
layers.append(nn.Tanh())

self.net = nn.Sequential(*layers)

def forward(self, z):
    return self.net(z.view(z.size(0), z.size(1), 1, 1))

```

```

[24]: class UniversalCritic(nn.Module):
    def __init__(self, im_chan=3, base_channels=64):
        super().__init__()

    def block(in_c, out_c):
        return nn.Sequential(
            nn.Conv2d(in_c, out_c, 4, 2, 1),
            nn.BatchNorm2d(out_c),
            nn.LeakyReLU(0.2, inplace=True)
        )

    self.net = nn.Sequential(
        nn.Conv2d(im_chan, base_channels, 4, 2, 1),

```

```

        nn.LeakyReLU(0.2, True),

        block(base_channels, base_channels*2),
        block(base_channels*2, base_channels*4),
        block(base_channels*4, base_channels*8),

        nn.Conv2d(base_channels*8, 1, 4, 1, 0),
        nn.Flatten(),
        nn.Sigmoid()
    )

    def forward(self, x):
        return self.net(x)

```

```

[25]: def generator_loss(fake_pred):
        return F.binary_cross_entropy(fake_pred, torch.ones_like(fake_pred))

    def discriminator_loss(real_pred, fake_pred):
        real_loss = F.binary_cross_entropy(real_pred, torch.ones_like(real_pred))
        fake_loss = F.binary_cross_entropy(fake_pred, torch.zeros_like(fake_pred))
        return 0.5 * (real_loss + fake_loss)

```

```

[26]: def train_one_epoch(G, D, loader, optim_G, optim_D, device, latent_dim=256):
    G.train(); D.train()
    lossG, lossD = 0, 0

    for real in tqdm(loader, desc="Training", leave=False):
        real = real.to(device)
        bs = real.size(0)

        z = torch.randn(bs, latent_dim, device=device)
        fake = G(z).detach()

        optim_D.zero_grad()
        loss_d = discriminator_loss(D(real), D(fake))
        loss_d.backward()
        optim_D.step()

        z = torch.randn(bs, latent_dim, device=device)
        fake = G(z)

        optim_G.zero_grad()
        loss_g = generator_loss(D(fake))
        loss_g.backward()
        optim_G.step()

```

```

        lossG += loss_g.item()
        lossD += loss_d.item()

    return lossD/len(loader), lossG/len(loader)

```

```

[27]: import warnings
      from PIL import Image

      warnings.filterwarnings("ignore", category=UserWarning, module="PIL.Image")

      latent_dim = 256
      target_size = (128,128)
      batch_size = 16
      epochs = 50

      data_path='/content/DLCV_Lab_Dataset/Human Faces Dataset/Real Images'

      dataset = CustomImageFolder(root=data_path, target_size=(128,128))
      loader = DataLoader(dataset, batch_size=batch_size, shuffle=True, num_workers=2)

      # G = UniversalGenerator(latent_dim=latent_dim, target_size=target_size).
      #   ↳to(device)
      G = UniversalGenerator(latent_dim=256, target_size=target_size).to(device)

      D = UniversalCritic().to(device)

      optim_G = torch.optim.Adam(G.parameters(), lr=2e-4, betas=(0.5, 0.999))
      optim_D = torch.optim.Adam(D.parameters(), lr=2e-4, betas=(0.5, 0.999))

      history_G = []
      history_D = []

      for epoch in range(1, epochs + 1):
          d, g = train_one_epoch(G, D, loader, optim_G, optim_D, device, latent_dim)
          history_D.append(d)
          history_G.append(g)

          print(f"Epoch {epoch:03d} | D Loss: {d:.4f} | G Loss: {g:.4f}")

```

Epoch 001 | D Loss: 0.1163 | G Loss: 5.8133

Epoch 002 | D Loss: 0.3198 | G Loss: 3.6828

Epoch 003 | D Loss: 0.2583 | G Loss: 4.1174

Epoch 004 | D Loss: 0.2545 | G Loss: 4.0936

Epoch 005 | D Loss: 0.2940 | G Loss: 3.6813

Epoch 006 | D Loss: 0.3135 | G Loss: 3.6159

Epoch 007 | D Loss: 0.3098 | G Loss: 3.6225

Epoch 008 | D Loss: 0.2925 | G Loss: 3.6493

Epoch 009 | D Loss: 0.2766 | G Loss: 3.8684

Epoch 010 | D Loss: 0.2630 | G Loss: 3.9173

Epoch 011 | D Loss: 0.2504 | G Loss: 4.1650

Epoch 012 | D Loss: 0.2579 | G Loss: 4.1945

Epoch 013 | D Loss: 0.2361 | G Loss: 4.0777

Epoch 014 | D Loss: 0.2256 | G Loss: 4.1370

Epoch 015 | D Loss: 0.2405 | G Loss: 4.1818

Epoch 016 | D Loss: 0.2042 | G Loss: 4.1046

Epoch 017 | D Loss: 0.2179 | G Loss: 4.2747

Epoch 018 | D Loss: 0.2062 | G Loss: 4.2627

Epoch 019 | D Loss: 0.2168 | G Loss: 4.1957

Epoch 020 | D Loss: 0.2136 | G Loss: 4.2125

Epoch 021 | D Loss: 0.2030 | G Loss: 4.2817

Epoch 022 | D Loss: 0.2094 | G Loss: 4.3951

Epoch 023 | D Loss: 0.1976 | G Loss: 4.1544

Epoch 024 | D Loss: 0.1954 | G Loss: 4.2279

Epoch 025 | D Loss: 0.1882 | G Loss: 4.3227

Epoch 026 | D Loss: 0.2042 | G Loss: 4.1078

Epoch 027 | D Loss: 0.1857 | G Loss: 4.4525

Epoch 028 | D Loss: 0.2283 | G Loss: 4.1423

Epoch 029 | D Loss: 0.1805 | G Loss: 4.0761

Epoch 030 | D Loss: 0.1570 | G Loss: 4.2532

Epoch 031 | D Loss: 0.1914 | G Loss: 4.4337

Epoch 032 | D Loss: 0.1739 | G Loss: 4.2490

Epoch 033 | D Loss: 0.1714 | G Loss: 4.6043

Epoch 034 | D Loss: 0.1363 | G Loss: 4.5463

Epoch 035 | D Loss: 0.1360 | G Loss: 4.7216

Epoch 036 | D Loss: 0.5994 | G Loss: 4.9217

Epoch 037 | D Loss: 0.1672 | G Loss: 3.8265

Epoch 038 | D Loss: 0.1708 | G Loss: 4.6787

Epoch 039 | D Loss: 0.1077 | G Loss: 4.6572

Epoch 040 | D Loss: 0.1080 | G Loss: 4.9746

Epoch 041 | D Loss: 0.2326 | G Loss: 5.0307

Epoch 042 | D Loss: 0.0960 | G Loss: 4.6374

Epoch 043 | D Loss: 0.0975 | G Loss: 4.9894

Epoch 044 | D Loss: 0.1533 | G Loss: 5.4836

Epoch 045 | D Loss: 0.1052 | G Loss: 5.1909

Epoch 046 | D Loss: 0.1744 | G Loss: 5.5153

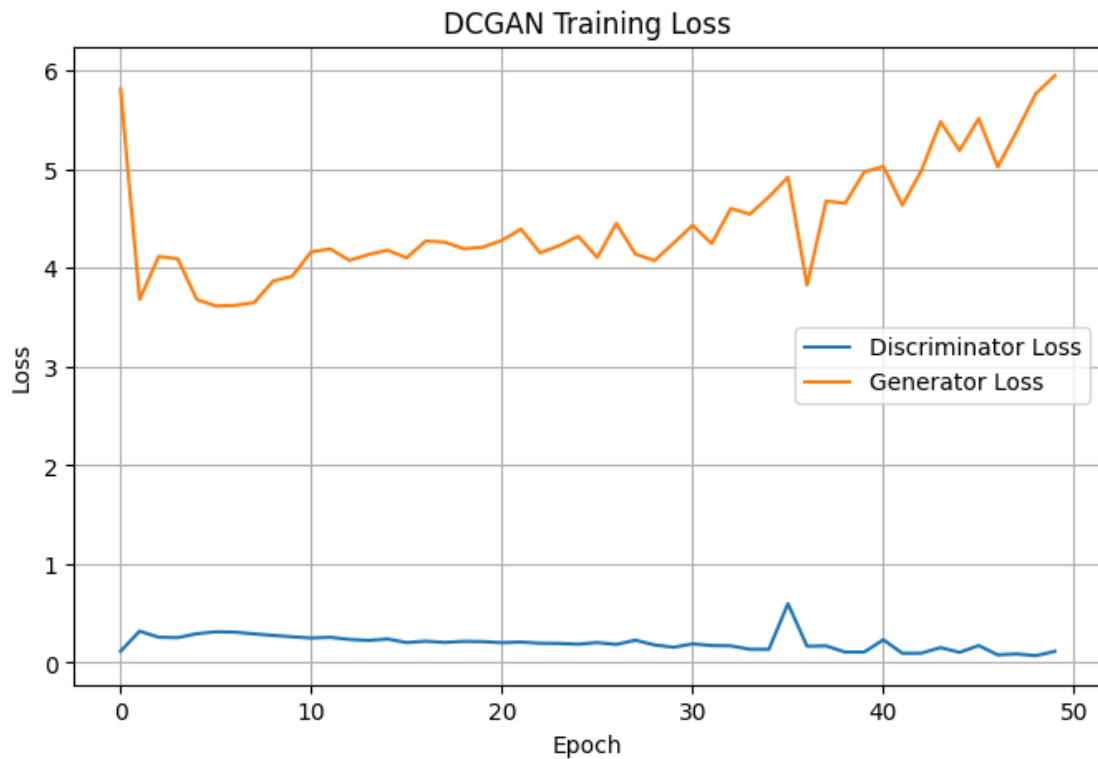
Epoch 047 | D Loss: 0.0800 | G Loss: 5.0253

Epoch 048 | D Loss: 0.0890 | G Loss: 5.3890

Epoch 049 | D Loss: 0.0715 | G Loss: 5.7694

Epoch 050 | D Loss: 0.1159 | G Loss: 5.9513

```
[28]: plt.figure(figsize=(8,5))
plt.plot(history_D, label="Discriminator Loss")
plt.plot(history_G, label="Generator Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("DCGAN Training Loss")
plt.legend()
plt.grid(True)
plt.show()
```



```
[29]: def denorm(x):
        return (x + 1) / 2
```

```
[30]: import matplotlib.pyplot as plt
import torchvision.utils as vutils

@torch.no_grad()
def generate_new_faces(G, num_images=16, latent_dim=256, device="cuda",
    ↪ save_path=None):
```

```

G.eval()

# Sample new latent vectors + these produce NEW synthetic faces
z = torch.randn(num_images, latent_dim, device=device)

fake_imgs = G(z)

# Convert to [0,1] for visualization
fake_imgs = denorm(fake_imgs)

plt.figure(figsize=(10, 10))
plt.axis("off")
plt.title("Generated New Synthetic Faces")
plt.imshow(
    vutils.make_grid(fake_imgs.cpu(), nrow=4).permute(1, 2, 0)
)
plt.show()

if save_path:
    vutils.save_image(fake_imgs, save_path, nrow=4)

return fake_imgs

```

```

[31]: @torch.no_grad()
def generate_new_face(G, latent_dim=256, device="cuda"):

    G.eval()

    # Sample one latent vector
    z = torch.randn(1, latent_dim, device=device)

    fake = G(z) # shape: (1, 3, H, W)
    fake = denorm(fake)[0] # shape: (3, H, W)

    img = fake.cpu().permute(1, 2, 0).numpy() # (H, W, 3)

    img = img.clip(0, 1)

    plt.figure(figsize=(5,5))
    plt.imshow(img)
    plt.axis("off")

```

```
plt.title("Generated Synthetic Face")  
plt.show()  
  
return img
```

```
[32]: img = generate_new_face(G, latent_dim=256, device=device)
```

Generated Synthetic Face

